

Name: Aziz Sayyad

Roll No: 381069

Batch: A3

Assignment No: 3

Problem Statement:

Perform text cleaning, perform lemmatization (any method), remove stop words (any method), label encoding. Create representations using TF-IDF. Save outputs.

Objective:

- To perform text cleaning by removing special characters, punctuation, and converting text to lowercase.
- To remove stop words from text using NLTK's predefined English stop word list.
- To apply lemmatization using NLTK's WordNet Lemmatizer to reduce words to their base forms.
- To encode categorical text labels into numerical values using Label Encoding.
- To create TF-IDF feature representations from the preprocessed text and save the final output to a CSV file.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/macOS
- Kernel: Python 3.x
- Tools: Google Colab / Jupyter Notebook / Anaconda
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: NLTK (stopwords, WordNetLemmatizer), re, Pandas, Scikit-learn (LabelEncoder, TfidfVectorizer)

Theory:

Text preprocessing is a fundamental step in any NLP pipeline. Raw text data is unstructured and contains noise such as punctuation, special characters, uppercase letters, common filler words, and inflected word forms. Cleaning and normalizing this text ensures that downstream models receive consistent, meaningful input.

Text Cleaning:

Text cleaning involves standardizing raw text by converting all characters to lowercase (to treat 'Python' and 'python' as the same word) and removing non-alphabetic characters such as punctuation, numbers, and special symbols using regular expressions. This reduces noise and vocabulary size.

Stop Word Removal:

Stop words are frequently occurring words in a language (e.g., 'is', 'the', 'with', 'for') that carry little semantic meaning for most NLP tasks. Removing them reduces feature dimensionality and helps models focus on content-bearing words. NLTK provides a corpus of 179 English stop words accessible via `nltk.corpus.stopwords`.

Lemmatization:

Lemmatization maps each word to its canonical dictionary form (lemma) using morphological analysis and the WordNet lexical database. For example, 'running' -> 'run', 'better' -> 'good'. Unlike stemming, lemmatization always produces valid English words, making it preferable when interpretability matters.

Label Encoding:

Label encoding converts categorical text labels into integer values. Scikit-learn's LabelEncoder assigns a unique integer to each class in alphabetical order (e.g., 'neutral' -> 0, 'positive' -> 1). This is required for most machine learning algorithms that expect numerical input for target variables.

TF-IDF Representation:

TF-IDF (Term Frequency - Inverse Document Frequency) converts preprocessed text into a numerical matrix where each cell represents the importance of a word to a specific document relative to the entire corpus. Words appearing frequently in one document but rarely across others receive higher scores, making TF-IDF effective for distinguishing documents.

Methodology:

- Install and import required libraries: NLTK, re, Pandas, and Scikit-learn. Download NLTK resources: punkt, stopwords, wordnet, omw-1.4.
- Create a sample dataset as a Pandas DataFrame with four text documents and corresponding 'positive' or 'neutral' labels.
- Text Cleaning: Define a function to convert text to lowercase and strip all non-alphabetic characters using regex (re.sub). Apply to the 'text' column to create a 'clean_text' column.
- Stop Word Removal: Load NLTK's English stop word set. Define a function to split each cleaned text and filter out stop words. Apply to create a 'no_stopwords' column.
- Lemmatization: Initialize WordNetLemmatizer. Define a function to split each word and apply lemmatize(). Apply to the 'no_stopwords' column to create a 'lemmatized_text' column.
- Label Encoding: Apply LabelEncoder on the 'label' column to generate an 'encoded_label' column with integer values (neutral=0, positive=1).
- TF-IDF Vectorization: Fit TfidfVectorizer on the 'lemmatized_text' column to produce a sparse TF-IDF matrix. Convert to a Pandas DataFrame with vocabulary terms as column headers.
- Combine the original DataFrame with the TF-IDF DataFrame using pd.concat and save the final output to a CSV file using to_csv().

Use Cases:

- Use Case 1 - Sentiment Classification Pipeline: The complete preprocessing pipeline (cleaning, stop word removal, lemmatization, label encoding, TF-IDF) prepares text data for training supervised classifiers like Naive Bayes or Logistic Regression to predict sentiment in product or movie reviews.
- Use Case 2 - News Article Categorization: Text cleaning and TF-IDF representations are used to build document feature matrices for clustering or classifying news articles into categories such as sports, politics, and technology.
- Use Case 3 - Data Archiving and Reproducibility: Saving the fully preprocessed and vectorized output as a CSV enables reuse across multiple experiments without repeating the preprocessing steps, which is critical in large-scale NLP research workflows.

Advantages:

- End-to-end pipeline covers all major preprocessing steps in a single workflow.
- Stop word removal and lemmatization significantly reduce feature dimensionality, speeding up model training.
- Label encoding converts categorical targets to a format directly usable by Scikit-learn classifiers.
- TF-IDF automatically handles term weighting, reducing the influence of common, uninformative words.
- Saving output to CSV ensures reproducibility and easy sharing of preprocessed datasets.

Limitations:

- Text cleaning with regex may inadvertently remove meaningful characters (e.g., hyphens in compound words or apostrophes in contractions).
- NLTK's stop word list is fixed; domain-specific common words (e.g., 'data' in a data science corpus) are not removed automatically.
- WordNet Lemmatizer without explicit POS tags defaults to noun lemmatization, potentially missing verb or adjective forms.
- TF-IDF does not capture word order or semantic relationships between terms.
- Label encoding imposes an ordinal relationship between classes, which may mislead certain algorithms if classes have no natural order.

Applications:

- Email spam detection: Preprocessing and TF-IDF features feed classifiers to distinguish spam from legitimate emails.
- Customer review analysis: Cleaned and lemmatized review text with encoded sentiment labels trains sentiment classifiers for e-commerce platforms.
- Legal document processing: Text cleaning and TF-IDF help retrieve and rank relevant legal documents based on query terms.
- Healthcare NLP: Preprocessing clinical notes and encoding diagnosis labels enables automated disease classification models.
- Search engine indexing: Cleaned, stop-word-free, lemmatized text improves index quality and query matching accuracy.

Conclusion:

This assignment implemented a complete NLP text preprocessing pipeline using Python. Starting from raw text, we applied cleaning to remove noise, stop word removal to eliminate uninformative terms, and WordNet lemmatization to normalize word forms. Label encoding converted categorical sentiment labels to integers, and TF-IDF vectorization produced a numerical feature matrix capturing term importance. The final combined DataFrame was saved to a CSV for future use. This pipeline demonstrates how systematic preprocessing transforms unstructured text into structured, machine-readable input, forming the essential foundation for any text classification or information retrieval system.